AI OPERATIONS — MODULE 3 · LECTURE 2B

# Scaling Model Serving with KServe

A self-contained ~2.5-hour lecture + hands-on session: FastAPI Predictor • HPA Autoscaling • Multi-Node Load Testing • KServe InferenceService

SESSION PLAN

# Agenda — ~2.5 Hours, Tiered

| Time | Block | Format |
|---|---|---|
| 0:00 – 0:15 | The inference scaling problem | Lecture |
| 0:15 – 0:30 | The workload: serving the RF classifier from Lecture 2a | Lecture |
| 0:30 – 0:50 | Tier 1: Deployment + Service + HorizontalPodAutoscaler | Lecture + code |
| 0:50 – 1:00 | Break | — |
| 1:00 – 1:40 | Hands-on (Tier 1): deploy, load-test, watch it scale across nodes | Lab |
| 1:40 – 2:00 | The KServe InferenceService: what it automates | Lecture |
| 2:00 – 2:15 | Canary rollouts; scale-to-zero (conceptual) | Lecture |
| 2:15 – 2:30 | Case study: scaling an image classifier during peak traffic | Discussion |
| 2:30 – 2:50 | Hands-on (Tier 2, optional): install KServe, deploy an InferenceService | Lab (optional) |

PREREQUISITES

# Before You Start This Session

- Completion of Lecture 2a: a running 3-node minikube cluster.
- Enable the metrics-server addon (needed by the autoscaler): minikube addons enable metrics-server
- No GPU required. The predictor is a CPU-only scikit-learn model, same as Lecture 2a.
- This session is TIERED: Tier 1 (plain Kubernetes autoscaling) is required and guaranteed to work with just kubectl. Tier 2 (real KServe) is an optional extension — see Section 5 for why.
- This deck is self-contained: every concept is followed by a runnable code snippet or command, and the hands-on exercises (amber slides) reference the companion Jupyter notebook.

00

# The Inference Scaling Problem

**What this section covers**

- The cost of an always-on Deployment
- What we're building today

0.1 THE COST OF ALWAYS-ON

# Sized for Peak, Paying for Idle

**24/7**

an always-on Deployment sized for PEAK traffic runs continuously

actual traffic at 3am, weekends, and holidays is often near zero

**$$$**

wasted compute during every quiet period, every single day

*Today's answer: a Deployment that scales its replica count up and down automatically based on real request load — and, with real KServe, potentially to zero.*

0.2 WHAT WE'RE BUILDING

# The Same Model, Now Behind a Scalable REST API

- We serve the RandomForest classifier trained in Lecture 2a as a REST prediction endpoint.
- A HorizontalPodAutoscaler (HPA) watches CPU usage and adds/removes replica Pods automatically.
- We load-test it ourselves and WATCH those replicas appear — spread across the same 3-node cluster from Lecture 2a.

01

# The Workload: A REST Predictor

**What this section covers**

- Reusing Lecture 2a's model
- The KServe V1 inference protocol

1.1 EXPORT THE TRAINED MODEL

# Same Dataset, Best Hyperparameters from the Grid Search

```
model = RandomForestClassifier(
    n_estimators=200, max_depth=12,   # from Lecture 2a's grid
    random_state=42, n_jobs=1,
)
model.fit(X_train, y_train)

joblib.dump(
    {"model": model, "feature_columns": list(X.columns)},
    "model.joblib",
)
```

*Saving feature_columns alongside the model matters: JSON/dict key order isn't guaranteed, so the predictor needs an explicit column order to interpret incoming requests correctly.*

# A Standard Request/Response Shape

```
POST /v1/models/{model_name}:predict

# Request:
{"instances": [[0.1, -2.3, 0.5, ...], [1.2, 0.0, -0.8, ...]]}

# Response:
{"predictions": [0, 1]}
```

*We implement this exact shape with plain FastAPI in Tier 1, so switching to a real KServe InferenceService in Tier 2 requires zero client-side changes.*

1.3 PREDICTOR_APP.PY

# A FastAPI Implementation of the V1 Protocol

```
from fastapi import FastAPI, HTTPException
import joblib, os, socket

app = FastAPI()
_bundle = joblib.load(os.environ["MODEL_PATH"])
POD_NAME = os.environ.get("POD_NAME", socket.gethostname())
NODE_NAME = os.environ.get("NODE_NAME", "unknown")

@app.post("/v1/models/{model_name}:predict")
def predict(model_name: str, request: PredictRequest):
    predictions = _bundle["model"].predict(request.instances).tolist()
    return {
        "predictions": predictions,
        "served_by_pod": POD_NAME,     # <- lets us PROVE which pod answered
        "served_by_node": NODE_NAME,   # <- and which node it ran on
    }
```

*served_by_pod / served_by_node go beyond the minimal V1 schema on purpose — they're what makes today's scaling demonstration empirically verifiable, not just claimed.*

02

# Tier 1: Plain Kubernetes Autoscaling

**What this section covers**

- Deployment + Service + HorizontalPodAutoscaler
- Guaranteed to work — no extra platform installs

2.1 WHY START HERE

# This Is What KServe Automates For You

- A real KServe InferenceService, under the hood, creates roughly this same trio of objects.
- Writing them by hand first means Tier 2 (Section 5) feels like “the same thing, automated,” not magic.
- It's also simply more reliable in a classroom setting: no Istio, no Knative, no extra CRDs to install — just kubectl and your existing cluster.

# deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rf-predictor
spec:
  replicas: 1
  selector:
    matchLabels: {app: rf-predictor}
  template:
    metadata:
      labels: {app: rf-predictor}
    spec:
      containers:
      - name: predictor
        image: rf-predictor:latest
        env: [POD_NAME, NODE_NAME]   # Downward API, same as Lecture 2a
        resources:
          requests: {cpu: "250m", memory: "256Mi"}
          limits:   {cpu: "500m", memory: "512Mi"}
        readinessProbe: {httpGet: {path: /healthz, port: 8080}}
```

2.3 SERVICE

# service.yaml — A Stable Address, However Many Replicas Exist

```
apiVersion: v1
kind: Service
metadata:
  name: rf-predictor-svc
spec:
  type: NodePort
  selector:
    app: rf-predictor
  ports:
  - port: 80
    targetPort: 8080
    nodePort: 30080
```

*Clients (including our load-testing script) always talk to ONE address — the Service transparently load-balances across however many replicas the HPA has created.*

2.4 HORIZONTALPODAUTOSCALER

# hpa.yaml — The Autoscaling Logic

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: rf-predictor-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: rf-predictor
  minReplicas: 1
  maxReplicas: 6
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: {type: Utilization, averageUtilization: 50}
```

*Requires the metrics-server addon (minikube addons enable metrics-server) so the HPA has real CPU numbers to act on.*

# Three kubectl apply Commands

```
$ minikube addons enable metrics-server
$ kubectl apply -f deployment.yaml
$ kubectl apply -f service.yaml
$ kubectl apply -f hpa.yaml

$ kubectl get pods -o wide
$ kubectl get hpa
# TARGETS shows <unknown> for ~60s until metrics-server reports real data
```

03

# Hands-on: Load Testing & Observing Scale-Out

**What this section covers**

- Firing concurrent requests
- Watching replicas appear across nodes

3.1 LOAD_TEST.PY

# Concurrent Requests, Tracking Which Pod/Node Answered

```
import concurrent.futures, requests, random

def fire_one_request(url, n_features=20):
    payload = {"instances": [[random.uniform(-3,3) for _ in range(n_features)]]}
    resp = requests.post(url, json=payload, timeout=10)
    body = resp.json()
    return {"status": resp.status_code, "latency_ms": ...,
            "served_by_pod": body["served_by_pod"],
            "served_by_node": body["served_by_node"]}

with concurrent.futures.ThreadPoolExecutor(max_workers=concurrency) as pool:
    futures = [pool.submit(fire_one_request, url) for _ in range(n_requests)]
    results = [f.result() for f in concurrent.futures.as_completed(futures)]
```

3.2 RUN THE SWEEP

# Increasing Concurrency: 1 → 10 → 40

```
$ minikube service rf-predictor-svc --url
# -> e.g. http://192.168.49.2:30080

$ python3 load_test.py \
    --url http://192.168.49.2:30080/v1/models/rf-classifier:predict \
    --concurrency-levels 1 10 40 --requests-per-level 100

# In a SECOND terminal, watch scaling happen live:
$ kubectl get hpa -w
$ kubectl get pods -o wide -w
```

*Run the load test and the two watch commands simultaneously (three terminals) to see the correlation between rising concurrency and rising replica count in real time.*

# The Autoscaling Story, End to End

- At concurrency=1: 1 replica, low latency, all requests served by the same pod.
- At concurrency=40: the HPA's REPLICAS column rises (up to maxReplicas: 6) as average CPU crosses 50%.
- kubectl get pods -o wide shows NEW replicas landing on DIFFERENT nodes — the same scheduler behavior from Lecture 2a, now triggered automatically by load rather than manually by us.
- load_test.py's distinct_nodes column climbing toward 3 is the empirical proof: traffic is genuinely being served from multiple machines.

04

# What KServe Automates

**What this section covers**

- The InferenceService: same trio, one declaration
- storageUri vs. a custom container

# Wrapping the Same Container

```yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: rf-classifier
  annotations:
    serving.kserve.io/deploymentMode: RawDeployment
    serving.kserve.io/autoscalerClass: hpa
    serving.kserve.io/metrics: cpu
    serving.kserve.io/targetUtilizationPercentage: "50"
spec:
  predictor:
    minReplicas: 1
    maxReplicas: 6
    containers:
    - name: kserve-container
      image: rf-predictor:latest    # <- the SAME image from Tier 1
```

*RawDeployment mode tells KServe to generate a plain Deployment + Service + HPA — exactly the Tier 1 objects you just wrote by hand.*

# storageUri (Production) vs. Custom Container (This Classroom)

## storageUri (Recall Module 1)

- storageUri: "models:/rf-classifier/Production"
- Points at Module 1's MLflow Model Registry.
- Works because production clusters have real networked/object storage, reachable from any node.

## Custom Container (Today)

- containers: [{image: rf-predictor:latest, ...}]
- Model is baked directly into the image.
- Chosen because minikube's default storage is node-local — sidesteps that limitation entirely for a reliable classroom demo.

# A Deliberate Classroom Tradeoff

- Full KServe (Knative + Istio + cert-manager + KServe CRDs) supports true SCALE-TO-ZERO — terminating ALL replicas when idle.
- That install chain is heavy and version-sensitive; on a shared classroom laptop within a single lecture, it is a common source of failed demos, not learning.
- RawDeployment mode uses plain HPA (what you just deployed in Tier 1) — reliable, but its minReplicas floor means it CANNOT scale below 1. Scale-to-zero stays conceptual here (next section), demonstrated live only if your instructor has a stable pre-built environment.

05

# Scale-to-Zero & Canary Rollouts

**What this section covers**

- True scale-to-zero: the Knative-based path (conceptual)
- Canary traffic splitting

5.1 SCALE-TO-ZERO (CONCEPTUAL)

# What Full KServe Adds Beyond RawDeployment

- With Knative Serving installed, KServe can terminate an InferenceService's LAST replica entirely when idle — releasing 100% of its compute.
- The first request after idling incurs a “cold start”: pulling and starting a fresh container before it can serve.
- This is exactly why Lecture 1's minimal, multi-stage image sizes matter operationally: a smaller image pulls and starts faster, directly shortening cold-start latency.

# Splitting Traffic Between Model Versions

```
spec:
  predictor:
    canaryTrafficPercent: 10   # 10% of traffic to the new revision

# Works identically whether the InferenceService uses RawDeployment
# or full Knative-based serverless mode.
```

*Lets a new model version absorb a small fraction of live traffic before a full rollout — connects back to Module 1's Registry stages/aliases and previews Module 4's deployment strategies.*

06

# Case Study

**What this section covers**

- Scaling an image classification service during peak traffic

6.1 CASE STUDY

# A Campus Photo-Classification App: Quiet Nights, Lab-Session Spikes

1

### 1. Overnight

Low or no traffic. RawDeployment stays at minReplicas: 1; full Knative mode could scale to zero.

2

### 2. Session Starts

Concurrency rises sharply as students submit photos — exactly what load_test.py simulated.

3

### 3. HPA Reacts

CPU utilization crosses the 50% target; new replicas are scheduled across all 3 nodes.

4

### 4. After the Spike

Traffic drops; HPA scales back down toward minReplicas over the stabilization window.

✎ HANDS-ON EXERCISE — 2B

~90–110 MIN

# Deploy, Load-Test, and Watch It Scale Across Nodes

1. Open the companion notebook: Lecture2b_KServe_Scaling_Exercise.ipynb.
2. Train and export the model; sanity-check predictor_app.py locally with a real HTTP request.
3. Build the predictor image with minikube image build; load it onto all 3 nodes.
4. Apply deployment.yaml, service.yaml, and hpa.yaml; enable metrics-server; confirm kubectl get hpa reports real CPU numbers.
5. Run load_test.py at increasing concurrency; watch kubectl get hpa -w and kubectl get pods -o wide -w in parallel terminals.
6. Plot latency and distinct-node count vs. concurrency; confirm distinct_nodes > 1 at your highest concurrency level.
7. (Optional, Tier 2) Install KServe in RawDeployment mode; deploy inference-service.yaml; repeat the load test against it.

**Deliverable:** HPA replica-count evidence, pod/node spread evidence, the latency + node-spread plots, and (optional) a working InferenceService load-tested the same way.

WRAP-UP

# Serving & Scaling Cheat Sheet

| Task | Code |
| --- | --- |
| Enable autoscaling metrics | minikube addons enable metrics-server |
| Deploy the predictor | kubectl apply -f deployment.yaml -f service.yaml -f hpa.yaml |
| Get a reachable URL | minikube service <svc-name> --url |
| Watch autoscaling live | kubectl get hpa -w |
| Watch pod-to-node spread | kubectl get pods -o wide -w |
| KServe: RawDeployment mode | annotations: serving.kserve.io/deploymentMode: RawDeployment |
| KServe: check status | kubectl get inferenceservice <name> |
| Configure a canary | spec.predictor.canaryTrafficPercent: 10 |

MODULE 3 — KEY TAKEAWAY

# Infrastructure is just another thing you can reproduce and prove:

**A Job that scales across CPUs and nodes (2a) + a Deployment that scales across nodes under load (2b) = the same Kubernetes primitives, two AI workload shapes**

*Every plot in this lecture and the last came from commands YOU ran against YOUR cluster — that is the difference between understanding Kubernetes and reciting facts about it.*

*Next: Module 4 — Model Serving & Deployment Strategies (Canary, Blue-Green, Shadow deployments).*